SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence
COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

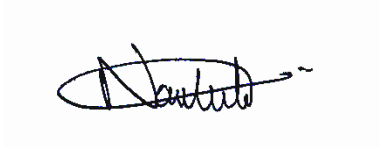
Annexure A

Analytical Problem Solving

Code of Conduct:

I **NANDHAKUMAARAN D (192511312)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Artificial Intelligence (AI)

Annexure A – Analytical Problem Solving: Answers

1. Water Jug Problem

Goal: Get exactly 2 gallons in the 4-gallon jug using a 4-gallon jug and a 3-gallon jug.

State representation: (x, y) where x = water in the 4-gallon jug (0–4) and y = water in the 3-gallon jug (0–3). Initial state (0, 0); goal state (2, y).

Operators (rules): Fill 4-gal jug; Fill 3-gal jug; Empty 4-gal jug; Empty 3-gal jug; Pour 3-gal into 4-gal until the 4-gal is full or the 3-gal is empty; Pour 4-gal into 3-gal similarly. **Solution**

Step	Action	State (4-gal, 3-gal)	Rule used
0	Start with both jugs empty	(0, 0)	Initial
1	Fill the 3-gallon jug from the pump	(0, 3)	Fill 3
2	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)	Pour 3→4
3	Fill the 3-gallon jug again	(3, 3)	Fill 3
4	Pour from 3-gal into 4-gal until the 4-gal is full (1 gallon goes in)	(4, 2)	Pour 3→4
5	Empty the 4-gallon jug onto the ground	(0, 2)	Empty 4
6	Pour the 2 gallons from the 3-gal jug into the 4-gal jug	(2, 0)	Pour 3→4

Result: The 4-gallon jug now holds exactly 2 gallons – goal reached in 6 moves.

Alternative (also 6 moves): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3). This reaches (2,3), which also satisfies the goal of 2 gallons in the 4-gallon jug.

2. Mars Rover – PEAS Analysis

The Mars Rover is an autonomous agent that explores the Martian surface, and collects and analyses soil/rock samples. It is described using the PEAS framework: **i. Percepts**

- Camera images (stereo/panoramic) of terrain, rocks and obstacles
- Sensor readings: wheel odometry, tilt/inclination, accelerometer, gyroscope, wheel-motor current
- Spectrometer, X-ray and chemical sensor data from soil and rock samples
- Temperature, wind, dust and radiation measurements; battery/solar power level
- GPS-like localisation/navigation data, and commands received from Earth (via orbiter relay)

ii. Operating Environment

Property	Characterisation
Observability	Partially observable – sensors only see the local surroundings
Agents	Single agent (no other competing agents)
Determinism	Stochastic – wheel slip, dust storms and uneven terrain make outcomes uncertain
Episodic / Sequential	Sequential – current moves affect future ones (power, position)
Static / Dynamic	Dynamic – weather, dust, lighting and temperature change while the rover acts
Discrete / Continuous	Continuous – positions, speeds, temperatures and time are continuous
Known / Unknown	Partly unknown – terrain and samples are new and unexplored

iii. Actions (Actuators)

- Move forward/backward, turn, steer and stop the wheels; climb or avoid obstacles
- Move the robotic arm; drill, scoop or grind rock and soil
- Deploy and point cameras and instruments; place samples in the on-board analysis lab
- Transmit data/images to Earth; receive commands
- Manage power: orient solar panels, switch to sleep/low-power mode, heat the electronics

iv. Performance Measure

- Number and scientific value of samples collected and analysed correctly
- Area explored / distance covered safely
- Safety: avoiding falls, getting stuck or damage; rover survival/lifetime
- Efficient use of energy, time and communication bandwidth
- Quality and amount of useful data successfully sent back to Earth

v. Suitable Agent Architecture

A utility-based agent with learning capability (a hybrid with reactive layers) is the most suitable. Reasons:

- Communication delay (4–24 minutes one way) means the rover cannot rely on Earth for real-time control – it must decide autonomously.
- A simple reflex agent is not enough, because the environment is partially observable and the rover needs to remember maps and past samples (model-based).
- It has multiple, sometimes conflicting goals (explore more vs. save power vs. stay safe vs. collect valuable samples). A utility function lets it trade these off, which a plain goal-based agent cannot do.
- A reactive layer handles emergencies instantly (e.g. stop on detecting a cliff), while a learning element improves terrain classification and navigation over time in an unknown environment.

3. 8-Queens Problem – Search Space and Formulation

Goal: place 8 queens on an 8×8 chessboard so that no two queens attack each other, i.e. no two share the same row, column or diagonal. The configuration given in the figure is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal.

A. Problem formulation – Incremental approach (preferred)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Better version: one queen per column in the leftmost n columns, with no queen attacking another.)
Initial state	Empty board – no queens placed
Actions	Add a queen to a square in the leftmost empty column such that it is not attacked by any queen already on the board
Transition model	Returns the board with the queen added to the specified square
Goal test	8 queens are on the board and none is attacked
Path cost	Not relevant – only the final configuration matters (each move can be given cost 1)

B. Alternative – Complete-state formulation

- States: all 8 queens on the board (one per column); the state can be written as a list [r1,…,r8] where ri is the row of the queen in column i.
- Initial state: any random arrangement of 8 queens (e.g. the figure's configuration).
- Actions: move one queen to another square in the same column.
- Goal test: no pair of queens attacks each other (number of attacking pairs h = 0). **C. Size of the search space**

Formulation	Number of states
Any 8 queens on any squares (64C8)	≈ 4.4 × 10^9 (1.8 × 10^14 sequences 64×63×…×57)
One queen per column, any row (8^8)	16,777,216
One queen per row and column (8! permutations)	40,320
Incremental, only non-attacking placements	2,057 states

Goal states: there are 92 solutions in total (12 distinct if rotations/reflections are treated as the same).

D. Exploring the search space using search strategies

- **Breadth-First Search (BFS):** explores all placements level by level; complete but needs huge memory because the frontier grows rapidly.

- **Depth-First Search with Backtracking:** place a queen column by column; if no safe row exists in a column, backtrack to the previous column and try the next row. Very memory-efficient and the standard way to solve 8-Queens.
- **Heuristic / Local search (Hill climbing, Simulated annealing, Min-conflicts):** start with the complete-state configuration, use h = number of attacking pairs of queens, and repeatedly move a queen to reduce h. Hill climbing can get stuck at local minima (random restarts or simulated annealing fix this).
- **Genetic Algorithm:** the state is a string of 8 digits (rows); fitness = number of non-attacking pairs (max 28); crossover and mutation produce better boards.

Applying to the given board: the first-column queen and last-column queen share a diagonal, so $h \geq 1$. Moving one of these queens within its column to a square that creates no conflict (using min-conflicts) reduces h; repeat until $h = 0$.

Sample solution (rows for columns 1–8): [1, 5, 8, 6, 3, 7, 2, 4] – a valid non-attacking arrangement.

4. OLA Cab Booking – Problem-Solving Agent

Type of agent: A **goal-based agent** (with utility-based selection of the cab type). The goal is to reach the destination; the customer's comfort/cost preferences (mini, micro, sedan, shared, prime…) act as a utility function that picks the best cab among the options that achieve the goal. It is a problem-solving agent that formulates the goal, finds options and chooses the best.

Problem formulation

- Initial state: customer at the pickup location, no cab booked.
- Goal state: customer dropped at the destination.
- Actions: enter locations, view cab types, select a cab type, confirm booking, track the cab, start the trip, end the trip, pay.
- Performance measure: low fare, short waiting/travel time, comfort and safety.

Pseudocode

```
function OLA-CAB-AGENT(percept) returns action
persistent: state, goal, options, preference

  state <- UPDATE-STATE(state, percept)   // pickup, drop, time, traffic, cab availability
if goal is empty then       goal <- FORMULATE-GOAL(state)       // reach destination D
from pickup P       preference <- GET-USER-PREFERENCE()  // budget, comfort, time,
shared/private

  if state.pickup = empty or state.drop = empty then
return ASK-LOCATIONS()

  options <- GET-AVAILABLE-CABS(state.pickup, state.drop)  // mini, micro, sedan, shared, prime
if options is empty then return NOTIFY("No cab available")

  best <- NULL;  bestUtility <- -infinity
for each cab in options do
     fare  <-  CALCULATE-FARE(cab,  distance,  time)
eta  <- ESTIMATE-ARRIVAL(cab)
     u    <- UTILITY(cab, fare, eta, comfort(cab), preference)
if u > bestUtility then
         best <- cab;  bestUtility <- u

  selected <- USER-CONFIRM(best, options)  // user may override with another type
booking    <-  BOOK-CAB(selected)      if  booking.status  =  FAILED  then  return
RETRY(options \ selected)

  TRACK-CAB(booking)                    // driver details, live location
if state.onBoard = true then NAVIGATE(state.pickup -> state.drop)
  if state.location = state.drop then
    COMPLETE-TRIP();   PAYMENT(fare);   GET-FEEDBACK()
goal <- empty   return action
```

5. Uniform Cost Search (UCS) – Least-Cost Path from S to G Edges read from the graph

From → To	Cost	Remark
S → A	1	
S → G	12	Direct route
A → B	3	
A → C	1	
B → D	3	
C → D	1	
C → G	2	
D → G	3	

UCS keeps a priority queue (frontier) ordered by path cost g(n) and always expands the cheapest node first. The goal test is applied when a node is selected for expansion.

Step-by-step execution

Step	Expand	Add / update in frontier	Frontier after (node:cost)
1	S (0)	A = 1, G = 12	A:1, G:12
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C:2, B:4, G:12
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (better than 12, so update)	D:3, B:4, G:4
4	D (3)	G via D = 3+3 = 6 (worse than 4, ignore)	B:4, G:4
5	B (4)	D via B = 4+3 = 7 (worse than 3, ignore)	G:4
6	G (4)	Goal test succeeds – stop	–

Order of expansion: S → A → C → D → B → G

Least-cost path: S → A → C → G with total cost = 1 + 1 + 2 = **4**

Comparison of other paths

Path	Cost	Optimal?
S → A → C → G	4	Yes
S → A → C → D → G	1+1+1+3 = 6	No
Path	**Cost**	**Optimal?**
S → A → B → D → G	1+3+3+3 = 10	No
S → G	12	No

Conclusion: The company should route packages along S → A → C → G, which costs 4 (fuel + time). UCS is complete and optimal because all edge costs are positive.